Tìm kiếm và suy luận

Từ chi phí đường đi tới kết luận logic

Khi quy tắc đã biết

Robot có bản đồ đầy đủ và thời gian đi trên từng đoạn.

Có cần huấn luyện lại để tìm đường không?

Ba câu hỏi tách biệt:

- Đường nào rẻ nhất?
- Đối thủ có thể làm gì?
- Kết luận nào đúng trong mọi tình huống hợp lệ?

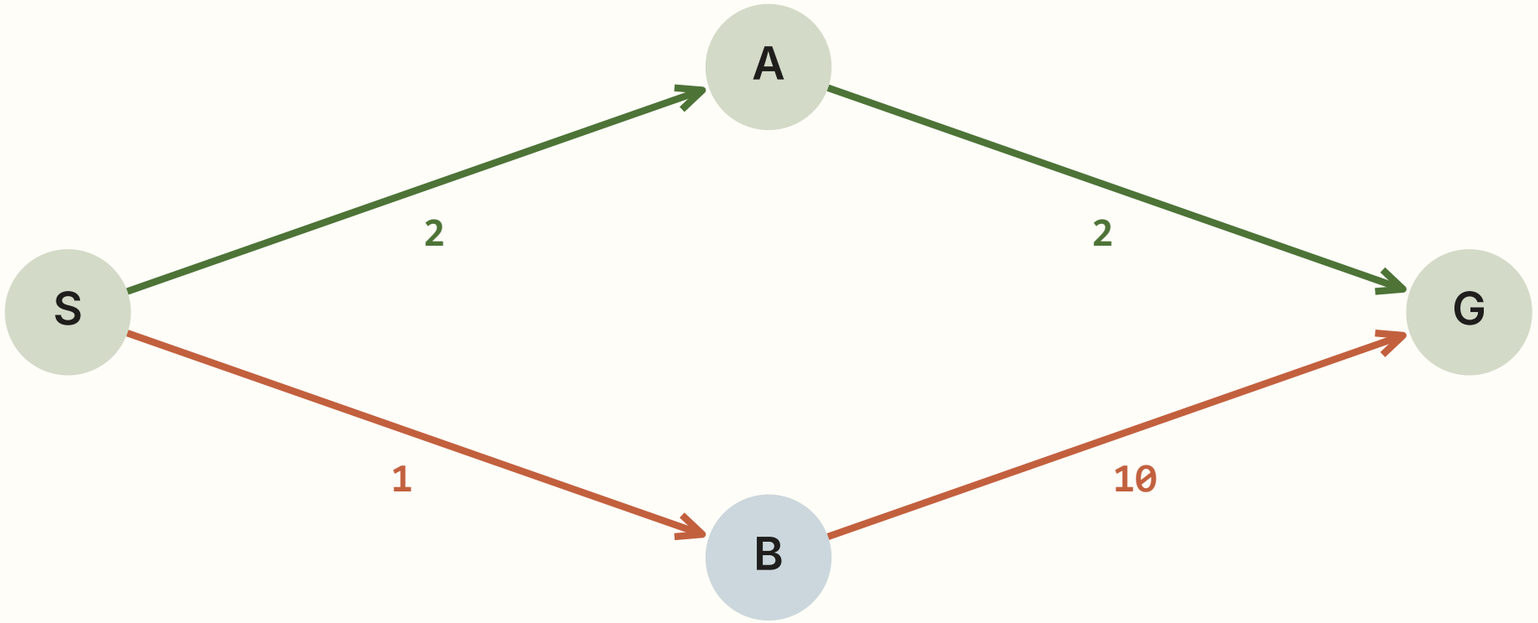
Mục tiêu

- Chọn BFS, Dijkstra hoặc A*.
- Tính A* bằng tay; biết khi nào heuristic làm mất lời giải tối ưu.
- Đọc cây minimax; hiểu alpha-beta và MCTS.
- Giải một bài CSP nhỏ và kiểm kết luận logic.

01

Trạng thái và chi phí

Ít cạnh chưa chắc ít thời gian



S–A–G: 4 giây

S–B–G: 11 giây

cả hai đường đều 2 cạnh; BFS trả đường qua B

BFS đếm cạnh nên chọn đường 11 giây thay vì đường 4 giây.

BFS trả lời câu nào?

```
bfs_answer = bfs_path("S", "G")
print(
    "BFS:", bfs_answer,
    "chi phí:", path_cost(bfs_answer)
)
```

KẾT QUẢ

```
BFS: ['S', 'B', 'G'] chi phí: 11
```

BFS tối thiểu số cạnh: hai đường đều 2 cạnh, B đứng trước nên được trả về.

Cạnh khác giá thì cần thuật toán tối ưu theo chi phí.

HỎI NHANH

Robot tới cùng một ô hai lần: một lần đã nhặt chìa khóa, một lần chưa. Có nên gộp hai lần đó thành một trạng thái không?

A Nên gộp, vì tọa độ đã mô tả đủ vị trí của robot

B Nên gộp, nếu dùng A* thay vì BFS

C Không gộp, nếu chìa khóa làm đổi các bước đi được về sau

D Không gộp, vì hai lịch sử khác nhau luôn là hai trạng thái

HỎI NHANH

Đồ thị có cạnh 1 giây và cạnh 2 giây. Một bạn muốn giữ BFS. Sửa thế nào để BFS cho ra đúng đường ít giây nhất?

A Giữ BFS, chỉ cộng đúng chi phí từng cạnh khi in đường

B Tách mỗi cạnh 2 giây thành hai cạnh 1 giây qua một đỉnh phụ

C Đưa các cạnh 1 giây lên trước trong danh sách kề

D Đánh dấu đã thăm lúc lấy ra khỏi hàng đợi, không phải lúc đẩy vào

02

Lời hứa của heuristic

A* cộng chi phí đã đi và chi phí ước lượng

là chi phí đã trả. là chi phí còn lại được ước lượng.

Cả hai đều có đơn vị giây trong bài này.

Dijkstra dùng . Greedy best-first chỉ dùng .

Hai điều kiện của heuristic

Admissible:

Không ước lượng quá chi phí còn lại thật.

Consistent, với mọi cạnh:

Chỉ có admissibility thì phải cho phép mở lại khi tìm được nhỏ hơn.

Lower bound từ bài toán nới lỏng

Lưới bốn hướng, mỗi bước ít nhất 1: bỏ tường làm bài dễ hơn, nên Manhattan không vượt chi phí thật.

Đổi luật đi (bước chéo, cổng dịch chuyển) thì phải chứng minh lại.

HỎI NHANH

Robot đi được 8 hướng như quân vua, mỗi bước, kể cả bước chéo, tốn 1 giây, và không có tường. Heuristic nào vừa admissible vừa sát nhất?

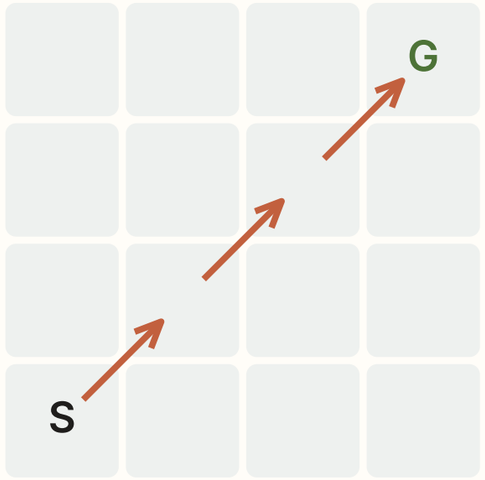
A , tức Manhattan

B $\sqrt{}$, tức Euclid

C

D

Ba bước chéo



3 bước chéo

Manhattan |Δx| + |Δy|	6	vượt
Euclid	4.24	vượt
max(|Δx|, |Δy|)	3	đúng bằng
min(|Δx|, |Δy|)	3	yếu khi lệch khác nhau

Đi 8 hướng: Manhattan và Euclid ước lượng quá, max(|Δx|, |Δy|) đúng bằng 3.

Một lượt A*

Sau khi mở S:

Đỉnh	g	h	f
A	2	2	4
B	1	10	11

A có nên được lấy ra trước.

Sinh G với , , .

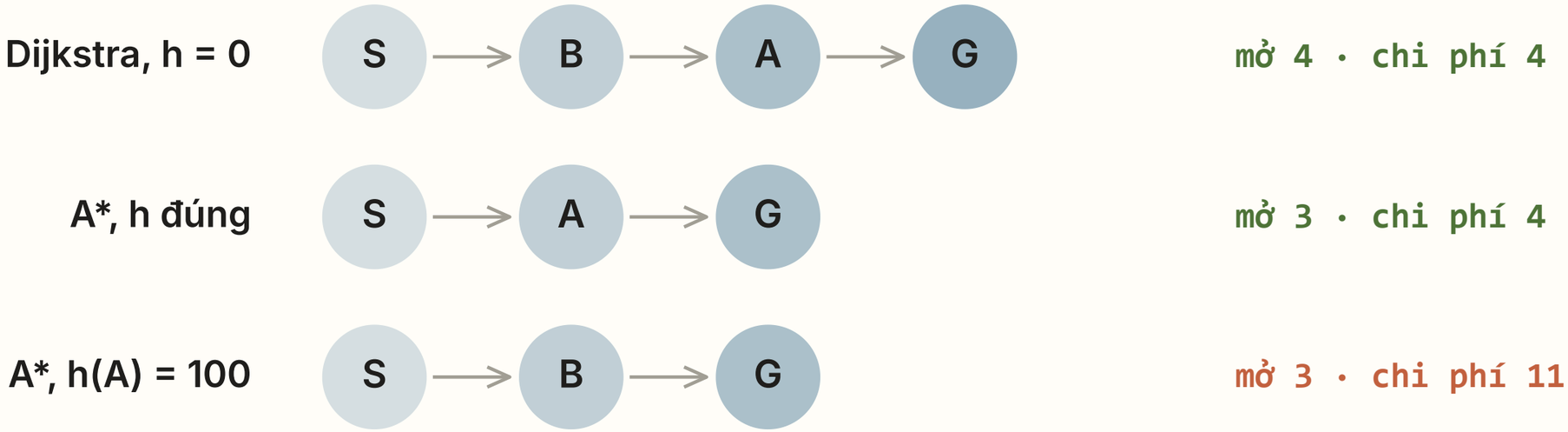
Lấy G ra khỏi heap, đường có chi phí tối ưu là 4 giây.

Đường rẻ hơn vẫn được cập nhật

```
_, cost, node = heapq.heappop(heap)
# bỏ mục heap đã lỗi thời
if cost != best_cost[node]:
    continue
expanded.append(node)
```

- Bỏ mục heap lỗi thời.
- Dừng khi lấy đích ra khỏi heap.
- Không khóa trạng thái ngay lúc đẩy vào heap.

Thứ tự mở đỉnh trong Dijkstra và A*



Heuristic tốt giúp A* mở ít đỉnh hơn Dijkstra; heuristic ước lượng quá làm A* trả đường sai.

Heuristic quá lớn làm mất tối ưu

KẾT QUẢ

```
Dijkstra: cost=4, path=['S', 'A', 'G'], mở=['S', 'B', 'A', 'G']
A*: cost=4, path=['S', 'A', 'G'], mở=['S', 'A', 'G']
A* h sai: cost=11, path=['S', 'B', 'G'], mở=['S', 'B', 'G']
```

Heuristic sai gán , nên không còn admissible.

Oracle vét cạn mọi đường đơn xác nhận chi phí tối ưu là 4.

HỎI NHANH

Sửa bản Dijkstra, tức , của bài để dừng ngay khi **sinh ra** G, thay vì khi lấy G ra khỏi heap. Trên đồ thị S, A, B, G, nó trả về chi phí bao nhiêu?

A 2

B 4

C 10

D 11

03

Model và ngân sách

Heuristic vượt một chút: cận vẫn còn

Nếu ở mọi , với :

Model đoán thấp: mở thêm đỉnh. Đoán cao: có thể bỏ đường tốt.

Weighted A*: với admissible; mở ít hơn, không quá .

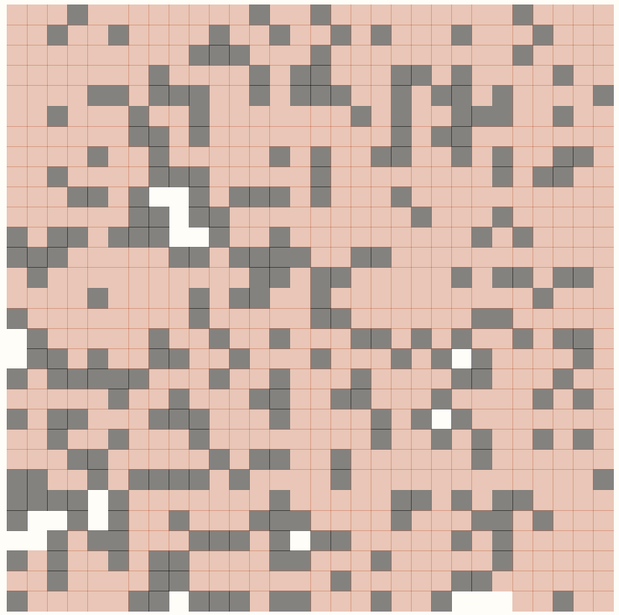
20 bản đồ , tường 30%

Cách tìm	Dài hơn tối ưu	Tệ nhất	Ô mở trung bình
Dijkstra	0/20	×1.000	594.8
A* Manhattan	0/20	×1.000	276.9
A* model (sai −30% đến +40%)	9/20	×1.103	142.4
Weighted A*	14/20	×1.310	115.0

Model và đều nằm dưới cận của mình: 1.4 lần và 2 lần.

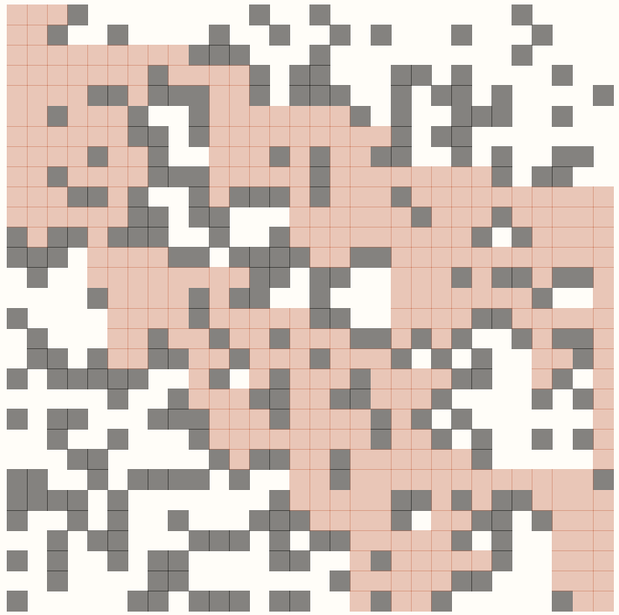
Ô đã mở trên một bản đồ

Dijkstra



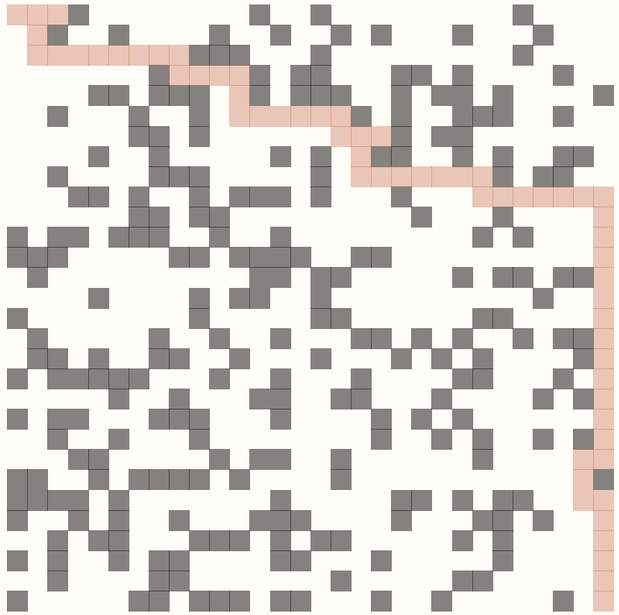
mở 613 · dài 58

A* Manhattan



mở 346 · dài 58

weighted A*, w = 2

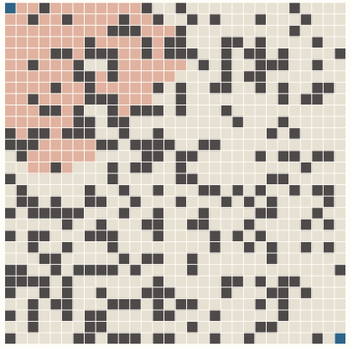


mở 63 · dài 60

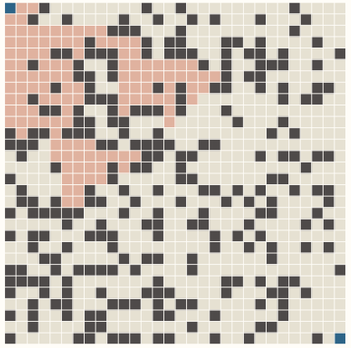
bản đồ seed 0 của bài; đường ngắn nhất dài 58; ô đen là tường, ô cam là ô đã mở

A* mở ít ô hơn Dijkstra mà vẫn tối ưu; weighted A* mở ít nhất nhưng có thể trả đường dài hơn.

Dijkstra và A* mở ô

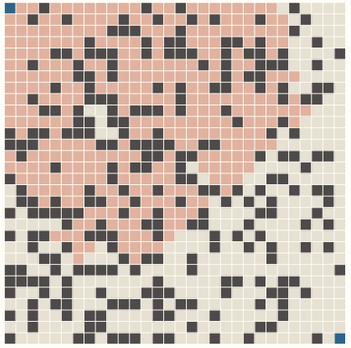


Dijkstra: đã mở 123 ô

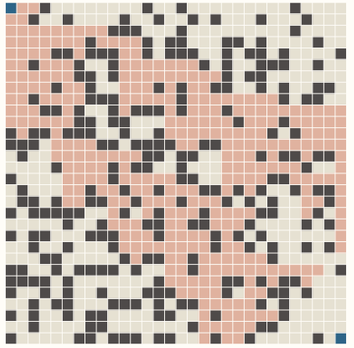


A* Manhattan: đã mở 123 ô

sau 123 lần lấy ô khỏi heap

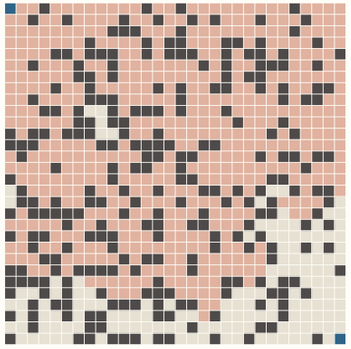


Dijkstra: đã mở 327 ô

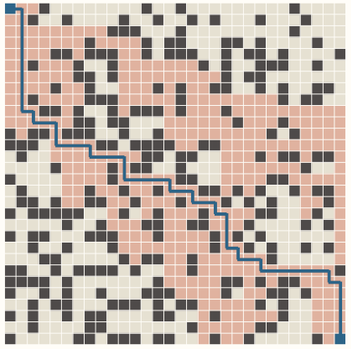


A* Manhattan: đã mở 327 ô

sau 327 lần lấy ô khỏi heap

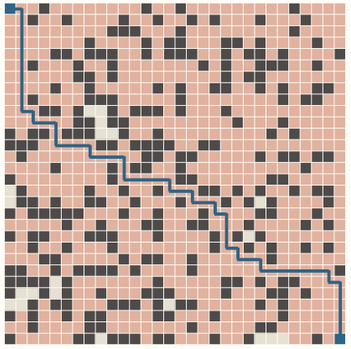


Dijkstra: đã mở 490 ô

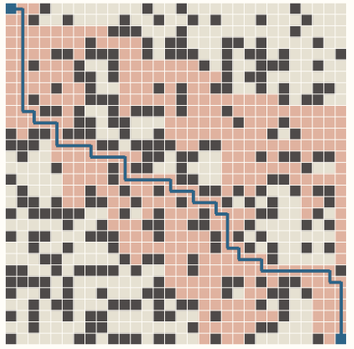


A* Manhattan: đã mở 346 ô

sau 490 lần lấy ô khỏi heap



Dijkstra: đã mở 613 ô



A* Manhattan: đã mở 346 ô

cả hai tìm đường dài 58 bước; Dijkstra mở 613 ô, A* Manhattan mở 346 ô

Dijkstra mở ô loang đều mọi hướng; A* mở lệch về phía đích, cùng độ dài đường đi.

HỎI NHANH

Weighted A* dùng Manhattan với , trả về một đường hợp lệ dài 54 bước, và chưa chạy thuật toán nào khác. Ta biết gì về độ dài tối ưu ?

A

B

C

D

HỎI NHANH

Đề yêu cầu in đúng độ dài đường ngắn nhất, sai là 0 điểm, và thời gian đủ cho Dijkstra. Theo bảng 20 bản đồ, nên dùng heuristic nào?

A Manhattan, vì nó mở ít đỉnh hơn Dijkstra mà vẫn tối ưu

B Model, vì nó mở ít đỉnh hơn Manhattan gần một nửa

C Weighted A* với , vì nó mở ít đỉnh nhất trong bảng

D Không dùng heuristic, vì heuristic nào cũng có thể sai

Hết giờ vẫn phải có đáp án

Anytime: lấy nhanh một đường bằng lớn, lưu lại; còn giờ thì giảm và tìm tiếp.

Hết giờ thì nộp đường tốt nhất đang có.

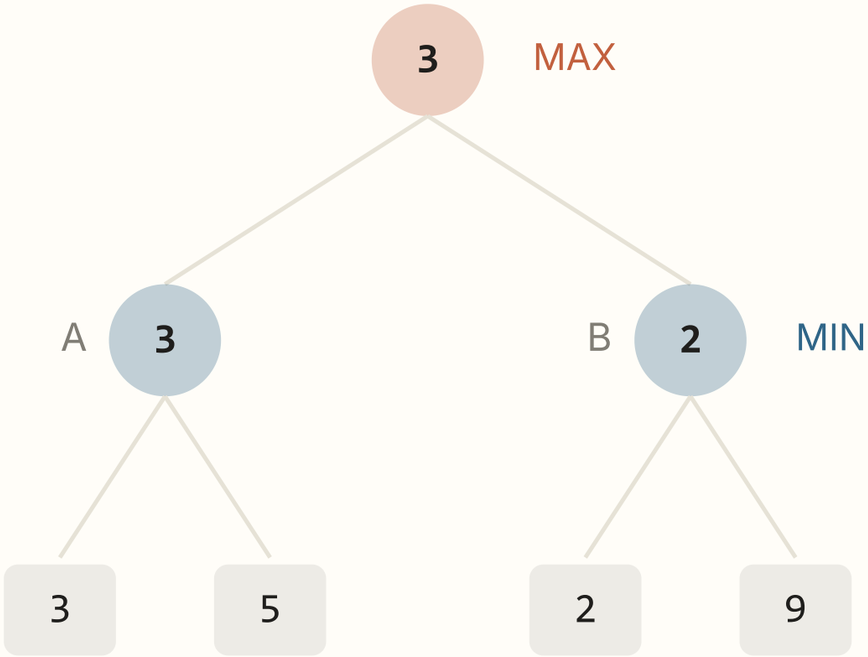
Beam search (bài 6.4): mỗi độ sâu giữ trạng thái, bộ nhớ cố định, có thể mất lời giải.

Đo số trạng thái mở mỗi giây trên test lớn nhất trước khi chọn.

04

Khi có đối thủ

Lá 9 chưa bảo đảm được 9



lá 9 lớn nhất cây nhưng MIN ở B chọn 2

MIN chọn 3 ở A, 2 ở B; MAX chọn A.

Minimax xét phản ứng tốt nhất của đối thủ

MAX lấy max. MIN lấy min. Lá terminal dùng utility.

Giả thiết: hai người, tất định, thông tin đầy đủ, tổng lợi ích bằng 0.

Đọc kết quả đệ quy

KẾT QUẢ

```
Giá trị hai nhánh: [3, 2]
Minimax: 3 chọn A: True
```

Điểm ở tất cả lá phải theo cùng góc nhìn.

Chọn lá lớn nhất đòi đối thủ hợp tác.
Minimax không giả định điều đó.

HỎI NHANH

Gốc là MAX, có hai nhánh MIN. Nhánh A có lá 4 và 6, nhánh B có lá 2 và 9. Vì sao MAX không chọn B để nhắm tới 9?

A MAX nên chọn B, vì 9 là lá lớn nhất cây

B MAX nên chọn B, vì trung bình lá của B là 5.5, hơn 5

C Vì ở B đối thủ sẽ chọn 2, còn A bảo đảm ít nhất 4

D Vì ở A đối thủ sẽ chọn 6, còn B chỉ bảo đảm 2

Alpha-beta: bỏ lá không đổi quyết định

A cho 3: MAX đã bảo đảm 3.

Ở B, lá đầu là 2: MIN ở B trả không quá .

Lá 9 không cần đọc.

Giá trị minimax giữ nguyên.

Xét nước tốt trước thì cắt nhiều hơn.

Cây đầy đủ: tới lá.

HỎI NHANH

Gốc là MAX, có ba nhánh MIN, xét từ trái sang phải. A có lá 6 và 4, B có lá 3 và 8, C có lá 5 và 1. Alpha-beta không cần đọc lá nào?

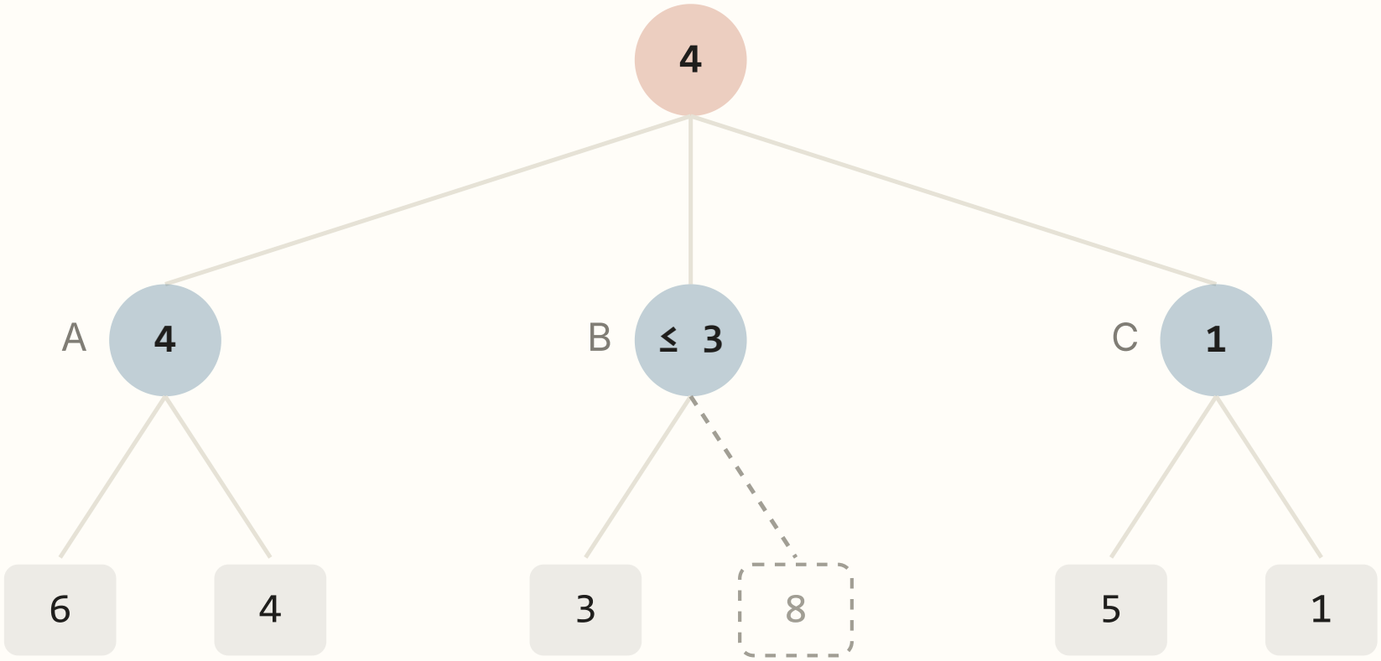
A Chỉ lá 8

B Lá 8 và lá 1

C Lá 4 và lá 8

D Không lá nào, vì nhánh nào cũng phải xét

Lá bị cắt



sau A, MAX đã có 4; lá 3 ở B đủ biết B không hơn 4, nên bỏ lá 8

Alpha-beta bỏ được lá 8 mà giá trị gốc vẫn là 4.

Cắt độ sâu và tầng chance

Cắt độ sâu với evaluation tạo giá trị ước lượng.

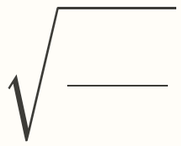
Evaluation có thể là model học từ ván đấu.

Có xúc xắc hoặc đối thủ ngẫu nhiên thì tầng chance lấy kỳ vọng.

Không dùng `min` thay xác suất của environment.

MCTS: dồn ngân sách vào nhánh hứa hẹn

Mỗi lượt, tại mỗi nút chọn con có



lớn nhất; mở rộng, đánh giá, cộng ngược.

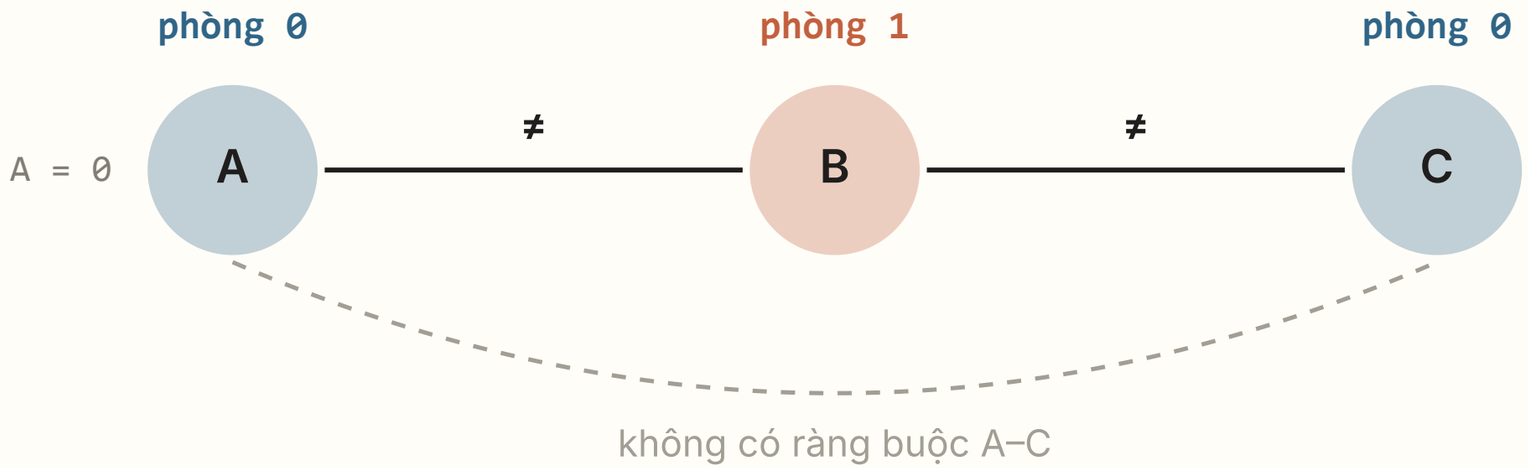
Số hạng sau ưu tiên nhánh ít được thử.

Hết ngân sách: nộp nước được thăm nhiều nhất.

05

CSP và logic

Xếp ba nhóm vào hai phòng



A ở phòng 0 buộc B ở phòng 1 và C ở phòng 0.

CSP tìm bộ gán hợp lệ

Biến: A, B, C.

Domain: .

Constraints: , , .

KẾT QUẢ

```
Nghiệm CSP: [{'A': 0, 'B': 1, 'C': 0}]
Khớp vét cạn: True
```

Quay lui bỏ nhánh vi phạm sớm. Oracle xét đủ 8 bộ gán.

Quay lui nhanh hơn mà không đổi bài

MRV: chọn biến còn ít giá trị hợp lệ nhất.

Forward checking: gán xong thì loại giá trị xung đột khỏi domain các biến chưa gán.

Domain rỗng thì bỏ nhánh ngay.

Không tự thêm ràng buộc đề không có.

HỎI NHANH

Thêm nhóm D phải khác phòng với cả A và B, vẫn chỉ có hai phòng, và . Quay lui gán theo thứ tự A, B, C, D, có forward checking. Vô nghiệm được phát hiện sớm nhất vào lúc nào?

A Chỉ khi gán tới D và đã thử cả hai phòng

B Sau khi gán C, vì C và D cùng nối với B

C Ngay sau khi gán B, vì domain của D đã rỗng

D Không phát hiện được, vì hợp lệ khi C cũng ở phòng 0

Domain thu hẹp sau mỗi lần gán

	ban đầu	gán A = 0	gán B = 1
A	0	0	0
B	0 1	1	1
C	0 1	0 1	0
D	0 1	1	∅

D khác A và khác B: hai phòng đã bị chiếm, domain của D rỗng trước khi gán C

Forward checking phát hiện D hết giá trị nên bỏ nhánh ngay.

Một nghiệm khác kết luận bắt buộc

: A ở phòng 0; : B ở phòng 0; : C ở phòng 0.

: R đúng trong **mọi** model của KB.

chỉ sai khi P đúng, Q sai: tương đương .

Kiểm model và phản ví dụ

KẾT QUẢ

```
Models của KB: [(True, False, True)]
KB suy ra R: True
Bỏ P, phản ví dụ cho R: [(False, True, False)]
```

Bỏ một giả thiết có thể làm kết luận mất hiệu lực.

KB vô nghiệm thì `all(...)` vẫn trả `True`: kiểm KB có nghiệm trước.

Tám thế giới, một model

P	Q	R	KB	KB bỏ P	
0	0	0	0	0	
0	0	1	0	0	
0	1	0	0	1	phản ví dụ: R sai
0	1	1	0	0	
1	0	0	0	0	
1	0	1	1	1	model duy nhất: R đúng
1	1	0	0	0	
1	1	1	0	0	

KB chỉ có một model và R đúng ở đó, nên KB suy ra R.

HỎI NHANH

Quay lui tìm được một cách xếp hợp lệ có C ở phòng 0. Đã chứng minh được C luôn ở phòng 0 chưa?

A Rồi, vì cách xếp đó thỏa mọi ràng buộc

B Rồi, vì quay lui duyệt theo một thứ tự cố định

C Chưa, phải tìm thêm một cách xếp nữa có C ở phòng 0

D Chưa, phải chắc rằng không model nào có C ở phòng 1

HỎI NHANH

Luật nói: nếu trời mưa thì sân ướt, tức . Quan sát thấy sân ướt. Với , có không?

A Có, vì luật nối thẳng M với U

B Không, vì model có sai và đúng vẫn thỏa KB

C Có, vì KB chỉ có đúng một model

D Không, vì KB mâu thuẫn nên không suy ra được gì

Khi đi thi

- Viết trạng thái và chi phí trước.
- Chấm tối ưu: A* admissible.
- Luôn giữ một lời giải hợp lệ.

06

Luyện đề

LUYỆN ĐỀ 1

Robot quay lên bắc, đích cách 6 ô thẳng về phía đông. Quay trái, quay phải hay tiến đều tốn 1, và là số ô còn lại. Chọn nhận định đúng về , và về kế hoạch bắt đầu bằng quay trái.

A admissible, kế hoạch tốn 7

B không admissible, kế hoạch tốn 9

C admissible, kế hoạch tốn 9

D không admissible, kế hoạch tốn 8

PHỎNG THEO CMU 10-601 · Exam 3 Practice · Q3.4

LUYỆN ĐỀ 2

Ở B có hai hành động: sang A được thưởng 50, giá trị hội tụ của A là 40; sang C được thưởng 30, giá trị của C là 100. Với , hành động tốt nhất ở B sau lần cập nhật đầu và khi hội tụ là gì?

A Sang C ngay từ đầu, và C khi hội tụ

B Sang A lúc đầu, rồi C khi hội tụ

C Sang A lúc đầu, và A khi hội tụ

D Sang C lúc đầu, rồi A khi hội tụ

PHỎNG THEO CMU 10-601 · Exam 3 Practice · Q4.3

Chốt ý

Search chọn khả năng cần xét; heuristic chỉ đổi thứ tự xét, cận cho biết mất bao nhiêu.

Minimax xét phản ứng của đối thủ; CSP tìm nghiệm; logic kiểm điều phải đúng.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook; đổi hoặc , dự đoán trước đường và số đỉnh mở, rồi kiểm lại bằng output.